

STRIP FOR PIXRUP

STRIPE_KEY=pk_test_51SLTZPKjD6pXy2vLFaWGYhhqMGKelGfcrV1iiDsIKAFndqJK5lzcQHly3Jh73x8eJ7Ly76v1UlbfPZ2icK7CLa6E00YcLPfUbk

STRIPE_SECRET=sk_test_51SLTZPKjD6pXy2vLjpSeEfDeSMx0dARChFm2UikVyzquF6HClolxmur2Z63zqCKpTMKjW4C1dJ7xGey0n40PmjAC00eqyWbQx4

STRIPE_WEBHOOK_SECRET=whsec_TVvkN4lZ65EexVVrrECtLBtARTCyrtI2

PRICE_MICRO=price_... # \$5 one-time

PRICE_STARTER=price_... # \$99/mo

PRICE_PRO=price_... # \$199/mo

PRICE_ENTERPRISE=price_... # \$499/mo

Webhook URL: <https://api.pixrup.com/stripe/webhook>

Webhook Events:

- checkout.session.completed
- invoice.paid

- customer.subscription.created
- customer.subscription.updated
- customer.subscription.deleted
- payment_intent.succeeded
- payment_intent.payment_failed

□ **1. Add these lines to your `.env` file**

Stripe Keys

```
STRIPE_KEY=pk_test_51SLTZPKjD6pXy2vLFaWGYhhqMGKelGfcrV1iiDsIKAFndqJK5lzcQHly3J  
h73x8eJ7Ly76v1UlbfPZ2icK7CLa6E00YcLPfUbk  
STRIPE_SECRET=sk_test_51SLTZPKjD6pXy2vLjpSeEfDeSMx0dARChFm2UikVyzquF6HClolxm  
ur2Z63zqCKpTMKjW4C1dJ7xGey0n40PmjAC00eqyWbQx4  
STRIPE_WEBHOOK_SECRET=whsec_TVvkN4lZ65EexVVrrECtLBtARTCyrtI2
```

Pricing IDs

```
PRICE_MICRO=price_...    # $5 one-time  
PRICE_STARTER=price_...  # $99/month  
PRICE_PRO=price_...      # $199/month  
PRICE_ENTERPRISE=price_... # $499/month
```

❓❓ 2. Install Stripe SDK

Run in terminal:

```
composer require stripe/stripe-php
```

❑❑ 3. Add these routes in **routes/web.php**

```
use App\Http\Controllers\StripeController;  
use App\Http\Controllers\StripeWebhookController;
```

```
Route::get('/checkout/{plan}', [StripeController::class, 'checkout']);  
Route::post('/stripe/webhook', [StripeWebhookController::class, 'handleWebhook']);
```

□ 4. Create **app/Http/Controllers/StripeController.php**

```
<?php
```

```
namespace App\Http\Controllers;
```

```
use Illuminate\Http\Request;
```

```
use Stripe\Stripe;
```

```
use Stripe\Checkout\Session;
```

```
class StripeController extends Controller
```

```
{
```

```
    public function checkout($plan)
```

```
    {
```

```
        Stripe::setApiKey(env('STRIPE_SECRET'));
```

```
        $priceId = match($plan) {
```

```
            'micro' => env('PRICE_MICRO'),
```

```
            'starter' => env('PRICE_STARTER'),
```

```
            'pro' => env('PRICE_PRO'),
```

```
'enterprise' => env('PRICE_ENTERPRISE'),  
default => abort(404),  
};
```

```
$session = Session::create([  
    'payment_method_types' => ['card'],  
    'mode' => in_array($plan, ['micro']) ? 'payment' : 'subscription',  
    'line_items' => [[  
        'price' => $priceId,  
        'quantity' => 1,  
    ]],  
    'success_url' => url('/success?session_id={CHECKOUT_SESSION_ID}'),  
    'cancel_url' => url('/cancel'),  
]);
```

```
return redirect($session->url);
```

```
}  
}
```

□ **5. Create `app/Http/Controllers/StripeWebhookController.php`**

```
<?php
```

```
namespace App\Http\Controllers;
```

```
use Illuminate\Http\Request;
```

```
use Illuminate\Support\Facades\Log;
```

```
use Stripe\Webhook;
```

```
class StripeWebhookController extends Controller
```

```
{
```

```
    public function handleWebhook(Request $request)
```

```
    {
```

```
        $payload = $request->getContent();
```

```
        $sigHeader = $request->header('Stripe-Signature');
```

```
        $secret = env('STRIPE_WEBHOOK_SECRET');
```

```
        try {
```

```
            $event = Webhook::constructEvent($payload, $sigHeader, $secret);
```

```
        } catch (\Exception $e) {
```

```
            Log::error('Stripe webhook error: ' . $e->getMessage());
```

```
            return response('Invalid', 400);
```

```
}
```

```
switch ($event->type) {  
    case 'checkout.session.completed':  
        Log::info('Checkout complete', $event->data->object);  
        break;  
    case 'invoice.paid':  
        Log::info('Invoice paid', $event->data->object);  
        break;  
    case 'payment_intent.succeeded':  
        Log::info('Payment succeeded', $event->data->object);  
        break;  
    case 'payment_intent.payment_failed':  
        Log::warning('Payment failed', $event->data->object);  
        break;  
    case 'customer.subscription.created':  
    case 'customer.subscription.updated':  
    case 'customer.subscription.deleted':  
        Log::info('Subscription event', $event->data->object);  
        break;  
    default:  
        Log::info('Unhandled event: ' . $event->type);  
}
```

```
    return response('Received', 200);  
  }  
}
```

6. Test It

- ❑ Go to your Stripe dashboard → **Developers** → **Webhooks**
- ❑ Click **Send test event** → choose `checkout.session.completed`
- ❑ If successful, you'll see a green **200 OK** in the log.

STEP 7 — TESTING FLOW (SANDBOX MODE)

Confirm environment

Make sure your Laravel app is using **test keys** (you are — `pk_test_...`, `sk_test_...`, `whsec_...`).

If you ever switch to live mode later, you'll just replace them with the live keys from Stripe.

Generate Stripe test checkout links

Open your terminal and run your app (e.g. `php artisan serve`).
Then open the browser and test each route:

Purpose	URL Example
Micro-Use (one-time \$5)	<code>http://localhost:8000/checkout/micro</code>
Starter Plan (\$99/mo)	<code>http://localhost:8000/checkout/starter</code>
Pro Plan (\$199/mo)	<code>http://localhost:8000/checkout/pro</code>
Enterprise (\$499/mo)	<code>http://localhost:8000/checkout/enterprise</code>

Each should redirect you to a Stripe-hosted checkout page (in test mode).

Use any **Stripe test card**, e.g.:

4242 4242 4242 4242 Exp: 12/34 CVC: 123 ZIP: 12345

Observe Webhook Delivery

After submitting the test checkout:

1. Go to <https://dashboard.stripe.com/test/webhooks>
2. Click your **Pixrup** webhook endpoint
3. Open the **Event deliveries** tab

□ You should see entries like:

```
checkout.session.completed payment_intent.succeeded invoice.paid
```

If any fail, check Laravel logs:

```
storage/logs/laravel.log
```

Simulate Subscription Events

In the same webhook dashboard:

1. Click **Send test event** → choose one from dropdown:
 - `customer.subscription.created`

- `customer.subscription.updated`
 - `customer.subscription.deleted`
2. Check that your Laravel terminal shows **200 OK** responses.
Each one will be logged inside `storage/logs/laravel.log`.
-

Simulate Payment Failures (Optional)

You can trigger failures with special test cards:

Scenario	Card Number
Payment failure	4000 0000 0000 9995
Insufficient funds	4000 0000 0000 0341
Expired card	4000 0000 0000 0069

You'll see the `payment_intent.payment_failed` event in Stripe logs.

Confirm in Stripe Dashboard

Go to **Payments** → **All Transactions**
You'll see all your test charges:

- ❑ “Succeeded” for good payments
- ❑ “Failed” for the bad ones

Then go to **Customers** → **Subscriptions** to verify subscription events fired correctly.

Clean Up (Optional)

After you’re satisfied:

- ❑ Delete old test customers from Dashboard → **Developers** → **Test data** → **Clear test data**
 - ❑ Keep your webhook endpoint active (you’ll reuse it when you switch to live).
-

❑ **Final Confirmation Checklist**

- ❑ Webhook returns 200 OK on Stripe dashboard
- ❑ Laravel `laravel.log` shows “Checkout complete”
- ❑ Stripe “Payments” section shows test transactions
- ❑ Subscriptions appear under “Customers”
- ❑ No “Failed” webhooks remaining

When all 5 checkmarks are green → Pixrup’s payment system is **production-ready**.

STEP 8 — GOING LIVE (Switch from Test Mode)

Go to Live Mode in Stripe Dashboard

- Navigate to <https://dashboard.stripe.com>
- In the left sidebar, you'll see a toggle:

Test mode ☐

- **Turn OFF test mode** (this switches to live environment).

You'll notice all test data disappears — now you're in production mode.

Get Your Live API Keys

1. Go to **Developers** → **API keys**
2. Copy your new **Publishable key** and **Secret key** — they'll start with:

```
pk_live_... sk_live_...
```

3. Go to your `.env` file on the server and replace the old test keys:

```
STRIPE_KEY=pk_live_XXXXXXX STRIPE_SECRET=sk_live_XXXXXXX
```

❑ **Do not share these keys** publicly — they control real money flow.

Recreate Live Products & Prices

Stripe separates “test” and “live” environments — your products from test mode won’t automatically carry over.

Do this:

1. Go to **Products** → **Create product** again.
2. Recreate your four live products exactly like before:
 - Pixrup Micro-Use — \$5 one-time
 - Pixrup Starter — \$99/month
 - Pixrup Pro — \$199/month
 - Pixrup Enterprise — \$499/month
3. Copy each **new live price ID** (`price_live_...`)

Update your `.env` file:

```
PRICE_MICRO=price_live_... PRICE_STARTER=price_live_...  
PRICE_PRO=price_live_... PRICE_ENTERPRISE=price_live_...
```

Recreate Your Webhook in Live Mode

Because webhooks are also separate between test/live:

1. Go to **Developers** → **Webhooks** → **Add endpoint**
2. Paste your same endpoint:

```
https://api.pixrup.com/stripe/webhook
```

3. Add these 7 events again:

```
checkout.session.completed invoice.paid  
customer.subscription.created customer.subscription.updated  
customer.subscription.deleted payment_intent.succeeded  
payment_intent.payment_failed
```

4. Save → Stripe will show a new **live webhook signing secret**:

```
whsec_live_XXXXXXXX
```

Copy it into `.env`:

```
STRIPE_WEBHOOK_SECRET=whsec_live_XXXXXXXX
```

Confirm Server Security (important)

Before launching payments publicly:

- ❑ Ensure your domain has **SSL (https)** (Hostinger does this automatically).
- ❑ Laravel `.env` file should have:

```
APP_ENV=production APP_DEBUG=false
```

- ❑ Verify firewall or `.htaccess` doesn't block `/stripe/webhook`.

Do One Real Test Payment

Make a \$5 purchase with a real card:

- ❑ Visit your live checkout route (e.g. `https://pixrup.com/checkout/micro`)
- ❑ Complete the payment
- ❑ Check your **Stripe Dashboard** → **Payments** → **Live**

If you see it appear instantly — you're officially live ❑

Notify Developer for Final Deployment

Tell your Laravel dev to:

- ❑ Clear cache (`php artisan config:clear && php artisan cache:clear`)
- ❑ Redeploy on your Hostinger VPS or production server
- ❑ Double-check that Stripe logs show **200 OK** for each webhook call

Summary — Your Live Launch Checklist

- ❑ Switched to **Live Mode**
- ❑ Recreated **Products**
- ❑ Recreated **Webhook**
- ❑ Replaced `.env` with live keys
- ❑ Confirmed **SSL** & `APP_DEBUG=false`
- ❑ Tested 1 real \$5 transaction

.